

Refatoração de Testes e Detecção de Test Smells

Relatório de Refatoração de Testes

Teste de Software
Gustavo Pereira de Oliveira
816374

02 de novembro de 2025

Análise de Test Smells

A suíte original apresenta três smells relevantes. O primeiro é a presença de lógica condicional dentro do teste `deve desativar usuários se eles não forem administradores`. O laço `for` combinado com `if/else` faz com que expectativas sejam executadas apenas em caminhos específicos, o que viola o padrão Arrange-Act-Assert e aumenta o risco de falsos positivos quando apenas parte do cenário falha.

O segundo smell é a fragilidade dos asserts no teste do relatório de usuários. A validação depende de uma string formatada integralmente; qualquer ajuste de formatação, mesmo sem alterar o comportamento, derruba o teste. Isso caracteriza um `Fragile Test`, que dificulta evoluções legítimas na camada de apresentação do relatório.

O terceiro smell está no teste que cobre o bloqueio para menores de idade. O uso de `try/catch` sem uma asserção obrigatória no fluxo feliz faz com que o teste passe silenciosamente caso a exceção não seja lançada. Esse padrão mascara regressões e prejudica a confiabilidade da suíte.

Processo de Refatoração

O cenário mais problemático envolvia a desativação condicional de usuários. A seguir apresentamos o trecho original e a versão refatorada.

Antes

```
for (const user of todosOsUsuarios) {
  const resultado = userService.deactivateUser(user.id);
  if (!user.isAdmin) {
    expect(resultado).toBe(true);
    const usuarioAtualizado = userService.getUserById(user.id);
    expect(usuarioAtualizado.status).toBe('inativo');
  } else {
    expect(resultado).toBe(false);
  }
}
```

Depois

```
test('deactivates non admin users', () => {
  const user = userService.createUser('Comum', 'comum@teste.com', 30);

  const result = userService.deactivateUser(user.id);
  const updatedUser = userService.getUserById(user.id);

  expect(result).toBe(true);
  expect(updatedUser.status).toBe('inativo');
});

test('does not deactivate admin users', () => {
  const admin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const result = userService.deactivateUser(admin.id);
  const storedAdmin = userService.getUserById(admin.id);

  expect(result).toBe(false);
});
```

```
    expect(storedAdmin.status).toBe('ativo');  
  });
```

A refatoração dividiu os cenários em testes independentes, cada um com seu próprio fluxo AAA. Com isso eliminamos a lógica condicional, tornamos o propósito de cada teste explícito e garantimos que falhas parciais sejam reportadas de forma clara.

Relatório da Ferramenta

A primeira execução do ESLint já destacou automaticamente os smells identificados. O plugin `eslint-plugin-jest` sinalizou expectativas condicionais, testes desabilitados e a ausência de asserts, reduzindo o esforço manual de inspeção.

```
npx eslint .
```

```
E:/test-smelly/test/userService.smelly.test.js
 44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning  Tests should not be skipped             jest/no-disabled-tests
 77:3  warning  Test has no assertions                  jest/expect-expect
```

```
✖ 6 problems (4 errors, 2 warnings)
```

Conclusão

A reescrita dos testes com foco em clareza e isolamento resultou em uma suíte mais confiável. A combinação de disciplina na aplicação do padrão AAA e o suporte de ferramentas de análise estática como o ESLint proporciona feedback rápido, facilita manutenções futuras e reforça a sustentabilidade do projeto.